# Geometriai algoritmusok 1

Szerző: Horváth Gyula
*horvath@inf.elte.hu*

Számos olyan gyakorlati számítási probléma van, amely megoldható olyan geometriai modellben, amelyben csak egyszerű objektumok, pontok, egyenesek, szakaszok, szögek, szögtartományok, poligonok szerepelnek. Ilyen feladatokat vizsgálunk. Nem foglalkozunk olyan feladatokkal, amelyek lebegőpontos aritmetikát igényelnének.

## 1.1. Alapfogalmak

**Pont:** $(x, y) \in \mathbb{R} \times \mathbb{R}$
**Szakasz**
Legyen $p_1$, $p_2$ pont. A $p_1$ és $p_2$ pont által meghatározott szakasz:
$\overline{p_1 p_2} = \{p = (x, y) : x = \alpha p_1.x + (1-\alpha) p_2.x,\ y = \alpha p_1.y + (1-\alpha) p_2.y),\ \alpha \in \mathbb{R},\ 0 \leq \alpha \leq 1\}$
Ha $p_1$ és $p_2$ sorrendje számít, akkor irányított szakaszról beszélünk, jele: $\overrightarrow{p_1 p_2}$.
**Egyenes**
Egyenes megadása:
1. $y = m x + b$ egyenlettel: azon $(x, y)$ pontok halmaza, amelyekre teljesül az egyenlet.
2. $a x + b y + c = 0$ egyenlettel.
3. Egyenes megadása két pontjával: $e(p_1, p_2)$

# 2. Alapműveletek

## 2.1. Irány

Adott az első síknegyeden az $A$ és a $B$ pont. Az origóból nézve az $A$ pont felé, a $B$ pont milyen irányban van: balra, jobbra, vagy egyirányban?
A válasz a pontokhoz tartozó szögek összehasonlításával megadható. Jelölje $\phi(O, A)$ az $A$-hoz, $\phi(O, B)$ a $B$hez tartozó szöget. $-\frac{\pi}{2} < \phi(O, A), \phi(O, B) \leq \frac{\pi}{2}$ és ebben a tartományban a tangens függvény szigorúan monoton növekvő, tehát
$\phi(O, A) < \phi(O, B)$ akkor és csak akkor, ha

$$\tan(\phi(O, A)) = \frac{A.y}{A.x} < \frac{B.y}{B.x} = \tan(\phi(Q, p_j))$$

Feltéve, hogy sem $A.x$, sem $B.x$ nem 0,

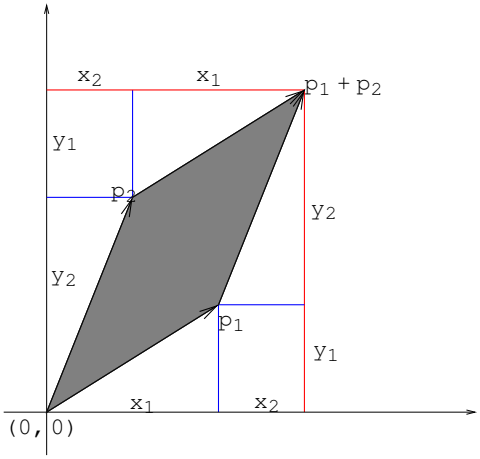
$$\phi(O, A) < \phi(O, B)$$

akkor és csak akkor, ha

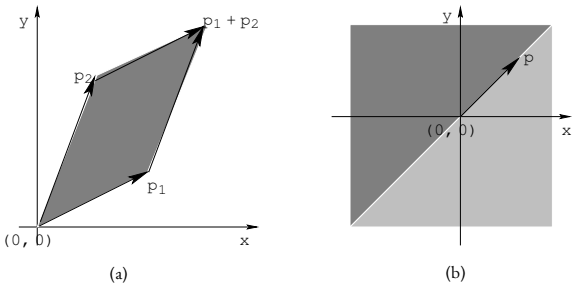
$$(A.y \cdot B.x) < (B.y \cdot A.x)$$
$$0 < B.y \cdot A.x - A.y \cdot B.x$$

Belátható, hogy az egyenlőtlenség akkor is igaz, ha a pontok nem az első síknegyedben vannak. A pontok viszonya kifejezhető az alábbi ábrán feltüntetett paralelogramma előjeles területével is.
Tekintsük a 3. ábrán látható $p_1$ és $p_2$ vektorokat.
A $p_1 \times p_2$ **keresztszorzat** a $(0, 0)$, $p_1$, $p_2$ és $p_1 + p_2 = (p_1.x + p_2.x,\ p_1.y + p_2.y)$ pontok által alkotott paralelogramma előjeles területeként értelmezhető.

$$\begin{aligned} p_1 \times p_2 &= \det \begin{pmatrix} x_1 & x_2 \\ y_1 & y_2 \end{pmatrix} \\ &= x_1 y_2 - x_2 y_1 \\ &= -p_2 \times p_1 . \end{aligned}$$




1. ábra. A paralelogramma előjeles területe: $T = (x_1 + x_2)(y_1 + y_2) - x_1y_1 - x_2y_2 - x_2y_1 - x_2y_1 = x_1y_1 + x_1y_2 + x_2y_1 + x_2y_2 - x_1y_1 - x_2y_2 - x_2y_1 - x_2y_1 = x_1y_2 - x_2y_1$




2. ábra. (a) A $p_1$ és $p_2$ vektorok keresztszorzata a paralelogramma előjeles területe. (b) A világos tartomány azokat a pontokat tartalmazza, amelyek a $p$-től órajárással egyező irányba esnek, a sötétebb pedig azokat, amelyek órajárással ellentétes irányba esnek $p$-től.

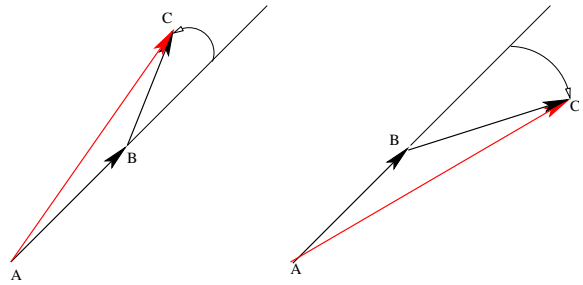
$p_1 \times p_2 > 0 \Leftrightarrow p_2$ az órajárással ellentétes irányban van $p_1$-hez képest.
$p_1 \times p_2 = 0 \Leftrightarrow$ a $(0, 0)$, $p_1$ és $p_2$ pontok egy egyenesre esnek (kollineárisak).
$p_1 \times p_2 < 0 \Leftrightarrow p_2$ az órajárás irányban van $p_1$-hez képest.

```
typedef struct {
  long long x,y;
  int azon;
} Pont;

int Irany(Pont A, Pont B){
  long long Kereszt=A.x*B.y-B.x*A.y;
  if (Kereszt <0)
    return -1;
  else if (Kereszt >0)
    return 1;
  else
    return 0;
}
```

## 2.2. Fordul

Még általánosabban, adott A, B és C pontok esetén A-ból B felé nézve a C pont milyen irányban van. Másképpen fogalmazva, adott két csatlakozó irányított szakasz, $\overrightarrow{AB}$ és $\overrightarrow{BC}$. Milyen irányba fordul $\overrightarrow{B-C}$ az $\overrightarrow{A-B}$-hez viszonyítva?



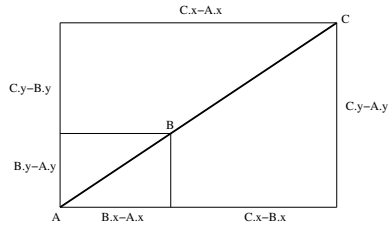

Csatlakozó szakaszok forgásiránya.

A válasz $(B-A) \times (C-A) = (B.x-A.x)(C.y-A.y) - (C.x-A.x)(B.y-A.y)$ keresztszorzat előjele alapján megadható.
$(B-A) \times (C-A)) > 0$ : $\overrightarrow{BC}$ balra fordul,
$(B-A) \times (C-A) = 0$ : $\overrightarrow{AB}$ és $\overrightarrow{BC}$ kollineárisak,
$(B-A) \times (C-A) < 0$ : $\overrightarrow{BC}$ jobbra fordul.

```
int Fordul(Pont A, Pont B, Pont C){
  long long Kereszt=(B.x-A.x)*(C.y-A.y)-(C.x-A.x)*(B.y-A.y);
  return (Kereszt>0)-(Kereszt<0);
}
```

### 2.3. Szakaszon

Eldöntendő, hogy `A`, `B`, `C` pontok esetén a `C` pont az `A` — `B` szakaszon van-e?

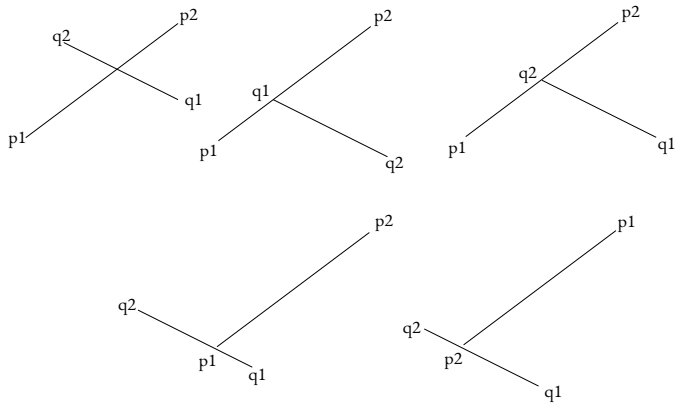



```
bool Kozte(Pont A, Pont B, Pont C){
  //Be: Fordul(A,B,C)=0
  //Ki: =A-B-C
  return abs(B.x-A.x) <= abs(C.x-A.x) &&
  abs(C.x-B.x) <= abs(C.x-A.x) &&
  abs(B.y-A.y) <= abs(C.y-A.y) &&
  abs(C.y-B.y) <= abs(C.y-A.y) ;
}
bool Szakaszon(Pont A, Pont B, Pont C){
  //Kimenet: csak akkor Igaz, ha a C pont a A-B szakaszon van
  return Fordul(A,B,C)==0 && Kozte(A,C,B);
}
```

# 3. Alapfeledatok

## 3.1. Szakaszpár metszi-e egymást?

Egy valódi metszés van, amikor a metszéspont nem végpontja egyik szakasznak sem, ls négy eset, amikor a metszéspont valamelyik szakasz végpont.

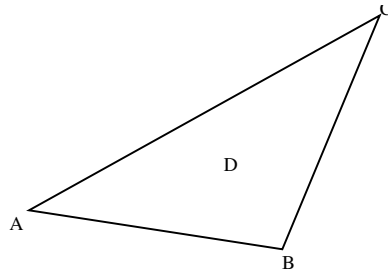



3. ábra. $\overline{p_1 - p_2}$ és $\overline{q_1 - q_2}$ szakaszpár metszésének öt lehetséges esete.

```
bool SzakaszParMetsz(Pont P1,Pont P2, Pont Q1,Pont Q2 ){
  int fpq1,fpq2,fqp1,fqp2;
  fpq1=Fordul(P1,P2, Q1);    fpq2=Fordul(P1,P2, Q2);
  fqp1=Fordul(Q1,Q2, P1);    fqp2=Fordul(Q1,Q2, P2);
  return fpq1*fpq2<0 && fqp1*fqp2<0 ||
  Szakaszon(P1,P2, Q1) || Szakaszon(P1,P2, Q2) ||
  Szakaszon(Q1,Q2, P1) || Szakaszon(Q1,Q2, P2);
}
```

## 3.2. Pont a háromszögben?

Döntsük el, hogy a `D` pont az $\triangle$ `(A, B, C)` háromszög belsejében van-e?




```
1   bool Haromszogben(Pont A,Pont B,Pont C, Pont D ){
2     return Fordul(A,B,D)==Fordul(B,C,D) && Fordul(B,C,D)==Fordul(C
3   }
```

**Definíció**

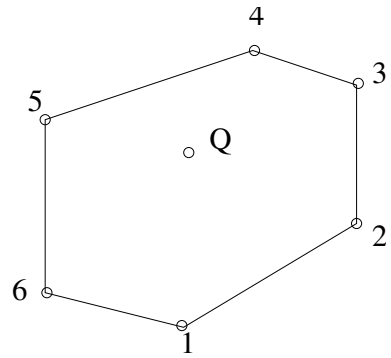
A `P = (p1, ..., pn)` zárt törtvonal (poligon) konvex, ha bármely két $p_i$ és $p_j$ pontjára teljesül. hogy a $\overline{p_i p_j}$ szakasz minden pontja a poligon belsejében van ($p_i$ és $p_j$ kivételével).

## 3.3. Poligon konvex-e?

Egy `P` poligon akkor és csak akkor konvex, ha bármely három egymást követő pontjára teljesül, hogy `Fordul(`$p_i$`,` $p_{i+1}$`,` $p_{i+2}$
`0`, ahol a `+1` és `+2` cirkulárisan értendő.

## 3.4. Pont konvex poligonban

Egy `Q` pont akkor és csak akkor van a `P` konvex poligonon belül, ha bármely két egymást követő pontjára teljesül, hogy `Fordul(`$p_i$`,` $p_{i+1}$`, Q) > 0`.

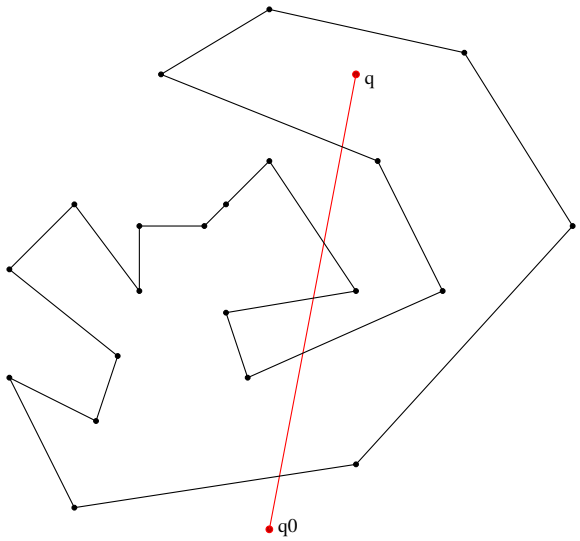


## 3.5. Pont és poligon helyzete

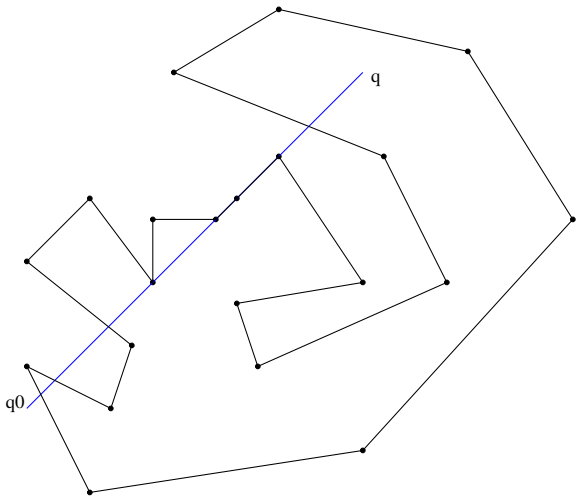
Adott a síkon egy zárt, nem-metsző poligon a csúcsainak $p_1, \ldots, p_n$ felsorolásával és adott egy $q$ pont. Eldöntendő, hogy a $q$ pont a poligon belsejéhez tartozik-e? A poligon valamely oldalára eső pont nem belső pontja a poligonnak.

**Megoldás**

Válasszunk egy olyan $q_0$ pontot, amely biztosan nem tartozik a poligon belsejéhez és a poligon egyetlen csúcsa sem esik a szak$q_0$ $q$ szakaszra, legfeljebb ha $q$ megegyezik valamelyik csúccsal. (Van ilyen pont.)
Ha a $\overline{q_0\ q}$ szakasz a poligon páratlan számú oldalát metszi, akkor $q$ belső pont, egyébként külső.

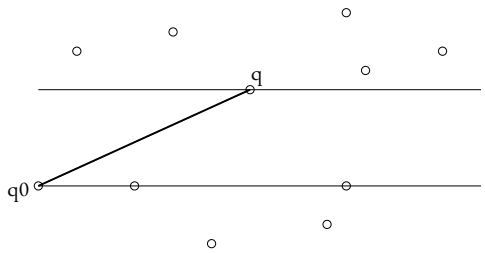



4. ábra. A $\overline{q_0\ q}$ szakasz a poligon páratlan sok oldalát metszi, tehát belső pont.




5. ábra. A poligonnak több csúcsa is a$\overline{q_0\ q}$ szakaszra esik.

A $q_0$ külső pont választása.
Legyen $q_0.y = \max\{p_i.y \mid p_i.y < q.y,\ i = 1, \ldots n\}$ és $q_0.x = \min\{p_i.x \mid i = 1, \ldots n\} - 1$.
Ha a poligonnak nincs olyan csúcspontja, amelynek $y$-koordinátája kisebb, mint $q.y$, akkor $q$ biztosan külső pont.
A $q0$ pont ilyen választása esetén legfeljebb a $q$ pont esik a poligon valamelyik oldalára. Ezt az esetet külön ellenőrizzük.
Egyébként, a poligon bármely $\overline{p_i\ p_{i+1}}$ oldalának akkor és csak akkor van közös pontja a $\overline{q_0\ q}$ szakasszal, ha $p_i$ és




6. ábra. A $q_0$ pont választása: $q_0.y = \max\{p_i.y | p_i.y < q.y, i = 1, \ldots n\}$ és $q_0.x = \min\{p_i.x | i = 1, \ldots n\} - 1$.

$p_{i+1}$ az $e(q_0, q)$ egyenes különböző oldalára esik és a $q_0$ és $q$ pont az $e(p_i, p_{i+1})$ egyenes különböző oldalára esik. Ez a feltétel a `Fordul` eljárás felhasználásával egyszerűen kiszámítható:

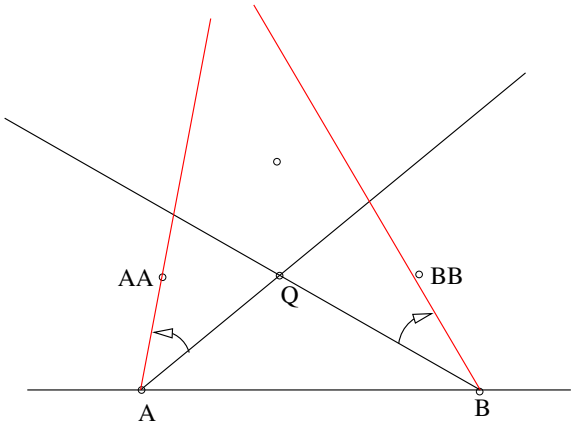
```
int Ponthelyzete(vector<Pont>& P, Pont Q){
  //Kimenet: -1 ha Q a poligon belsejében van,
  //          0 ha az oldalán,
  //          1 ha kivül van
  int y0,x0, i,ii,metsz=0;
  Pont Q0;//a külső pont
  y0=Q.y; x0=P[0].x;
  int n=P.size();
  for(int i=0;i<n;i++){
    if (P[i].y<Q.y && (P[i].y>y0 || y0==Q.y) )
    y0=P[i].y;
    if (P[i].x<x0) x0=P[i].x;
  }
  if (y0==Q.y)//Q kölső pont
  return 1;
  Q0.y=y0; Q0.x=x0-1;//Q0 a külső pont
  for(int i=0;i<n;i++){
    int ii=(i+1)% n;
    if (Szakaszon(P[i], P[ii], Q) ) return 0;
    if (Fordul(P[i],P[ii],Q0)*Fordul(P[i],P[ii],Q)<0 &&
    Fordul(Q0, Q, P[i])*Fordul(Q0, Q, P[ii]) <0 )
    metsz++;
  }
  if(metsz%2==0)
  return -1;
  else
  return 1;
}
```

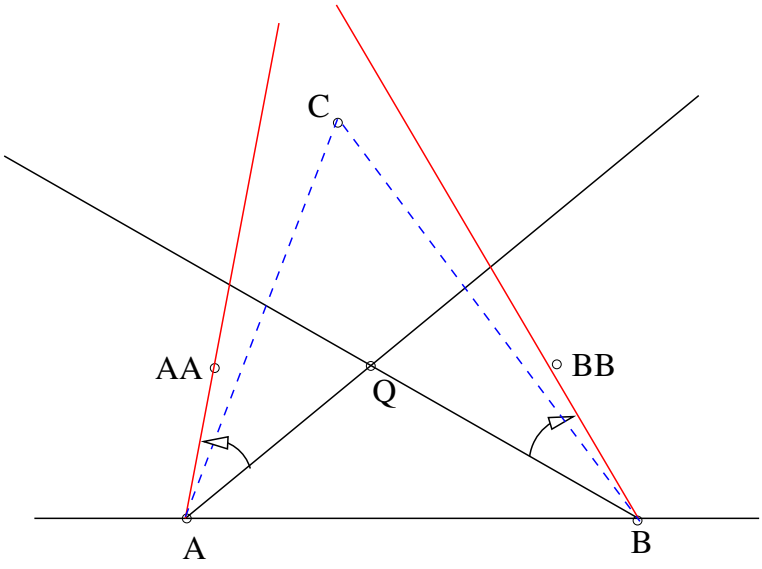
# 4. Feladat: IOI Válogató 2006

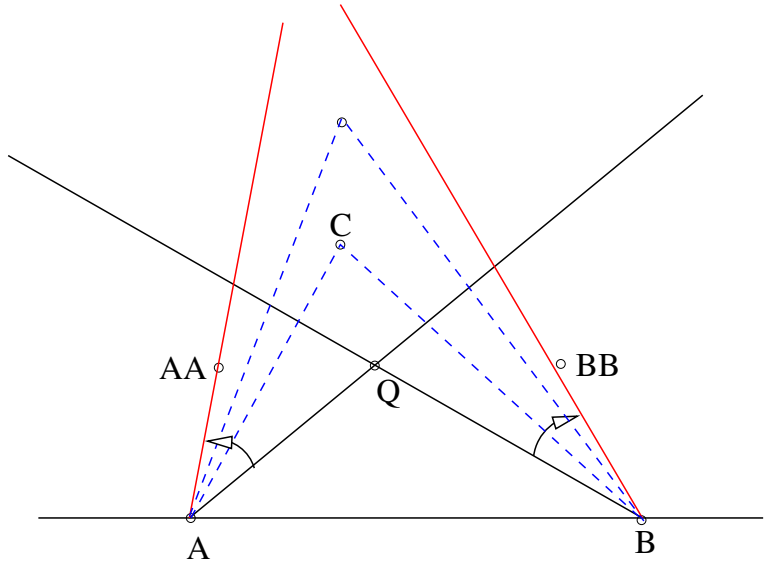
Adott a síkon egy `P` ponthalmaz és két kitüntetett pontja; `A` és `B`. Adott továbbá egy `Q` pont. Kiszámítandó a `P` ponthalmaz egy olyan `C` pontja, hogy a `Q` pont az $\triangle$`(A, B, C)` háromszög belsejében van (nem eshet az oldalára sem), és a `P` ponthalmaz egyetlen más pontja sem esik a háromszögbe (oldalára sem eshet).
Követelmény: `O(n)` futási idejű algoritmus.

## Megoldás

Határozzuk meg azt az `AA` pontot (ha van), amely $\overrightarrow{AQ}$ -tól balra az első, azt a `B` pontot (ha van), amely $\overrightarrow{BQ}$ -tól balra az első. Az `A`, `AA`, `Q`, `B` és `BB` pontok az ábrán látható módon meghatározzák azt a tartományt, amelybe eső pontok lehetnek megoldáspontok. Ha már megtaláltuk ez első ilyet, akkor a továbbiaknak már az $\triangle$`(A, B, C)` háromszögbe kell esniük.
Az algoritmus futási ideje a pontok számával lesz arányos.





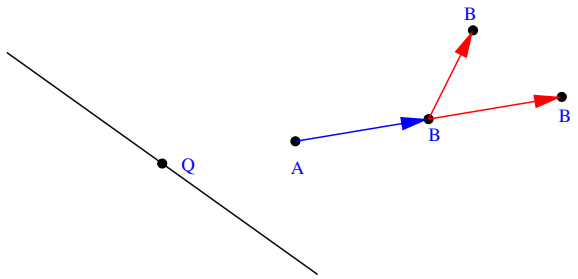

```
int Bekerit2(vector<Pont>& P, int A, int B, Pont Q){
  int n=P.size();
  Pont AA, BB, C;
  int FrAB,FrBQ,FrAQ,FrAC,FrBC;
  AA.azon=-1; BB.azon=-1; C.azon=-1;
  //AA és BB meghatározása
  for(int i=0;i<n;i++){
    if(P[i].azon==A|| P[i].azon==B) continue;
    FrAB= Fordul(P[A],P[B],P[i]);
    FrBQ= Fordul(P[B],Q,P[i]);
    FrAQ= Fordul(P[A],Q,P[i]);
    if (FrAB>=0 && FrBQ>=0 && FrAQ<=0)
    return -1;
    if (FrAB>0 && FrAQ>0 && FrBQ>0 &&
    (AA.azon==0 || Fordul(P[A],AA,P[i])<0) )
    AA=P[i];
    if (FrAB>0 && FrAQ<0 && FrBQ<0 &&
    (BB.azon==0 || Fordul(P[B],BB,P[i])<0) )
    BB=P[i];
  }
```

```
    // egy C meghatározása
    int FrAAA,FrBBB;
    for(int i=0;i<n;i++){
      if(P[i].azon==A|| P[i].azon==B) continue;
      if (AA.azon>0) FrAAA= Fordul(P[A],AA,P[i]);
      if (BB.azon>0) FrBBB= Fordul(P[B],BB,P[i]);
      FrAQ= Fordul(P[A],Q,P[i]);
      FrBQ= Fordul(P[B],Q,P[i]);
      if ((AA.azon==0||FrAAA<0)&&(BB.azon==0||
            FrBBB>0)&& FrAQ>0&&FrBQ<0){
        C=P[i];
        break;
      }
    }
    if(C.azon<0) return -1;

    //háromszög finomítás
    for(int i=C.azon+1;i<n; i++){
      if (P[i].azon==A || P[i].azon==B ||
      P[i].azon==AA.azon ||
          P[i].azon==BB.azon) continue;
      FrAB= Fordul(P[A],P[B],P[i]);
      FrAC= Fordul(P[A],C,P[i]);
      FrBC= Fordul(P[B],C,P[i]);
      if (FrAB>=0 && FrAC<=0 && FrBC>=0){
        C=P[i];
      }
    }
    return C.azon;
  }
```

## 4.1. Sarokpont szerint rendezés

Egy Q pontot a P ponthalmaz sarokpontjának nevezünk, ha van olyan Q-n átmenő $\epsilon$ egyenes, hogy a P ponthalmaznak nincs két olyan pontja, amely az $\epsilon$ egyenes nem azonos oldalára esne.
Legyen Q a $P = \{P_1, \ldots, P_n\}$ ponthalmaz bal-alsó sarokpontja. Definiáljuk a P halmazon azt a rendezési relációt, amely szerint $P_i < P_j$ akkor és csak akkor, ha $\phi(Q, p_i) < \phi(Q, p_j)$ vagy $\phi(Q, p_i) = \phi(Q, p_j)$ és $P_i$ közelebb van Q-hoz, mint $P_j$. $\phi(Q, p_i) < \phi(Q, p_j)$ akkor és csak akkor, ha $\texttt{Fordul}(Q, P_i, P_j) > 0$. Tehát rendezési relációt az alábbi függvény megvalósítja.




$A \leq B$

```
1  bool SarokRend(Pont Q, Pont A, Pont B){
2    return Fordul(Q,A,B)>0 || Fordul(Q,A,B)==0 && Kozte(Q,A,B);
3  }
4  //
5  void SarokRendez(vector<Pont>& P){
6    Pont Q;
7    Q.x=INT_MAX;Q.y=INT_MAX;
8    int n=P.size();
9    for(int i=0;i<n;i++){
10     if(P[i].x<Q.x || P[i].x==Q.x && P[i].y<Q.y)
11     Q=P[i];
12   }
13   sort(P.begin(), P.end(),[&Q](Pont A,Pont B)->bool
14     {return SarokRend(Q,A,B);});
15 }
16
17 void SarokRendez(vector<Pont>& P, Pont Q){
18 //Q sarokpont
19   sort(P.begin(), P.end(),[&Q](Pont A,Pont B)->bool
20     {return SarokRend(Q,A,B);});
21 }
```

# 5. Feladat: Pontok összekötése zárt, nem-metsző poligonná.

Adott a síkon `n` darab pont, amelyek nem esnek egy egyenesre. A pontok `(x, y)` koordinátáikkal adottak, amelyek egész számok. A pontokat a `1, ..., n` számokkal azonosítjuk. Kössünk össze pontpárokat egyenes szakaszokkal úgy, hogy olyan zárt poligont kapjunk, amelyben nincs metsző szakaszpár. Egy ilyen zárt, nem-metsző poligon megadható a pontok azonosítóinak egy felsorolásával: a felsorolásban egymást követő pontokat kötjük össze egyenes szakaszokkal, továbbá, az utolsót az elsővel is összekötjük.

## Bemenet

A `standard bemenet` első sora a pontok `n` $(3 < n < 1000)$ számát tartalmazza. A további `n` sor mindegyike két egész számot tartalmaz, egy pont `x` és `y` koordinátáit, $(-20000 \leq x, y \leq 20000)$. A pontok nem esnek egy egyenesre.

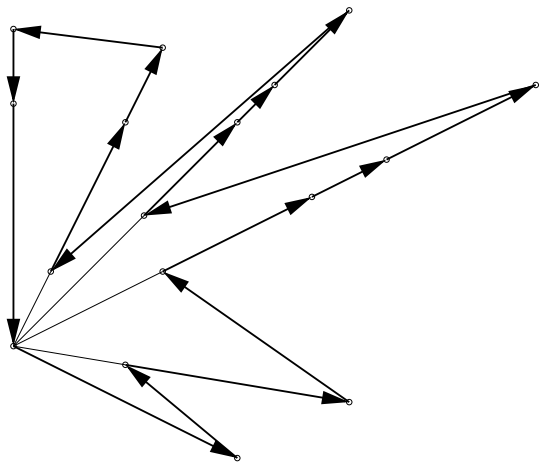
## Kimenet

A `standard kimenet` első sorába a nem-metsző poligont leíró sorozatot kell kiírni.

## Példa

| Bemenet | Kimenet |
|---|---|
| 6<br>2 0<br>1 4<br>0 2<br>3 2<br>2 4<br>2 6 | 3 1 4 5 6 2 |

## 5.1. A poligon probléma megoldása

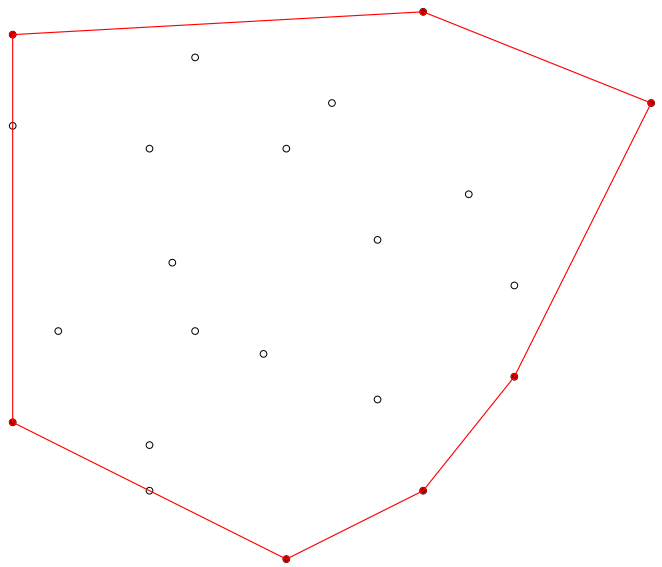


Rendezzük a ponthalmazt a bal-alsó sarokpont szerint. A rendezett sorrendben az egymást követő pontokat kötjük össze egyenes szakasszal, kivéve a legnagyobb szögben látszókat, azokat a rendezés szerint fordított sorrendben.

```
void Poligon(vector<Pont>& P){
  Pont Q;
  Q.x=INT_MAX;Q.y=INT_MAX;
  int n=P.size();
  for(int i=0;i<n;i++){
    if(P[i].x<Q.x || P[i].x==Q.x && P[i].y<Q.y)
    Q=P[i];
  }
  SarokRendez(P, Q);
  int ii=n-2;
  while(ii>1 && Fordul(Q,P[n-1],P[ii])==0)
    ii--;
  for(int i=0;i<=ii;i++)
    cout<<P[i].azon<<" ";
  for(int i=n-1;i>ii;i--)
    cout<<P[i].azon<<" ";
  cout<<endl;
}
```

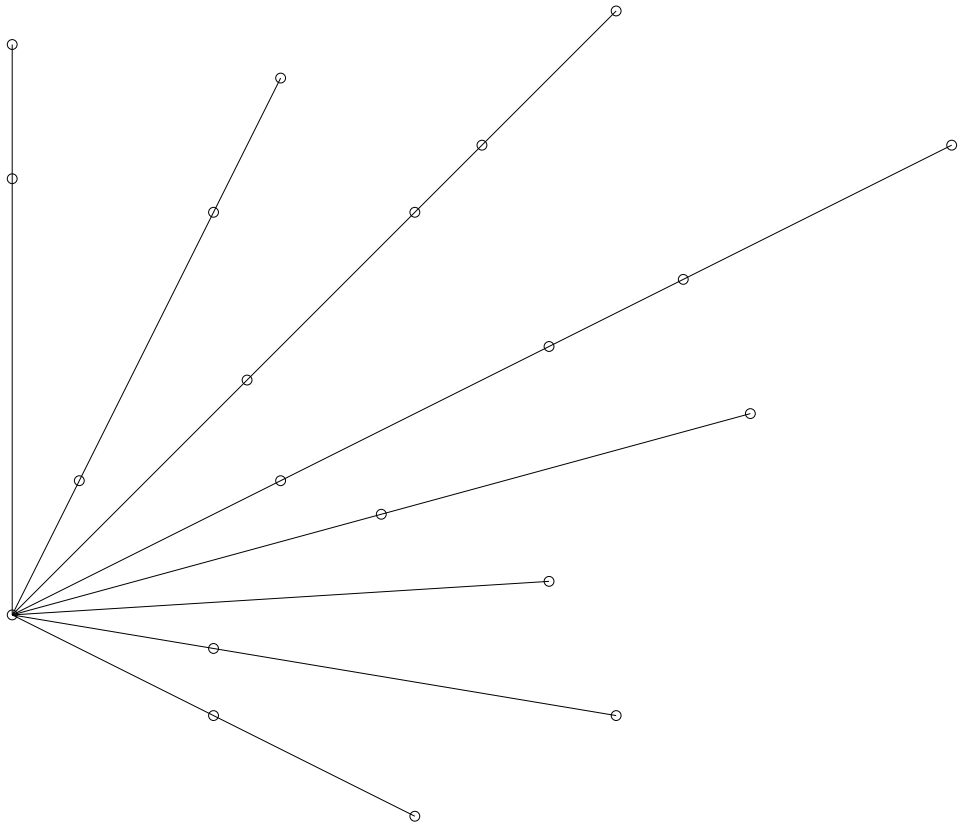
# 6. Ponthalmaz konvex burka

**Definíció**. Egy egyszerű (nem metsző) poligon konvex, ha bármely két belső pontját összekötő szakasz minden pontja a poligon belsejében, vagy határán van.
**Definíció**. Egy $P = \{p_1, \ldots, p_n\}$ ponthalmaz konvex burka az a legszűkebb $Q$ konvex poligon, amely a ponthalmaz minden pontját tartalmazza.
**Megoldás.** Első lépésként rendezzük a ponthalmazt a bal-alsó sarokpontra vett polárszög szerint, majd minden egye-

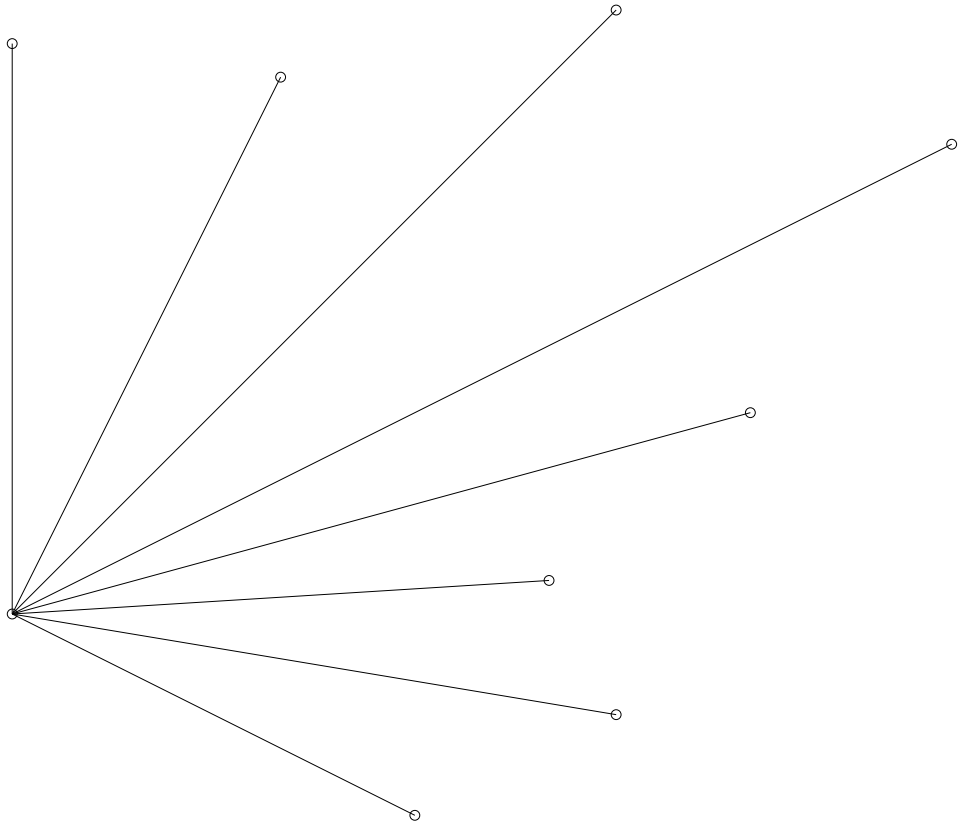


7. ábra. Ponthalmaz konvex burka.

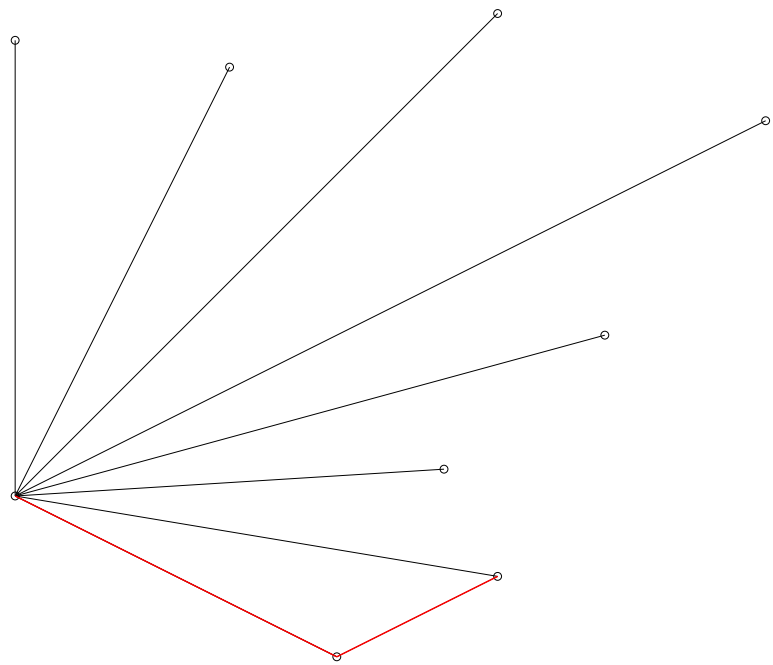
nesen csak a sarokponttól legtávolabbi pontot hagyjuk meg, a többit töröljük. Az így törölt pontok biztosan nem lehetnek csúcspontjai a konvex buroknak. Legyen $\langle q_1, \ldots, q_m \rangle$ az így kapott pontsorozat polárszög szerinti rendezésben. Mindaddig, amíg van három olyan cirkulárisan egymást követő $q_i$, $q_{i+1}$ $q_{i+2}$ pont, hogy $\overrightarrow{q_{i+1}\, q_{i+2}}$ nem balra fordul $\overrightarrow{q_i\, q_{i+1}}$-hez képest, hagyjuk el a $q_{i+1}$ pontot.
Az algoritmus futási ideje $O(n \lg n)$, ha a polárszög szerinti rendezést olyan algoritmussal valósítjuk meg amelynek futási ideje $O(n \lg n)$.

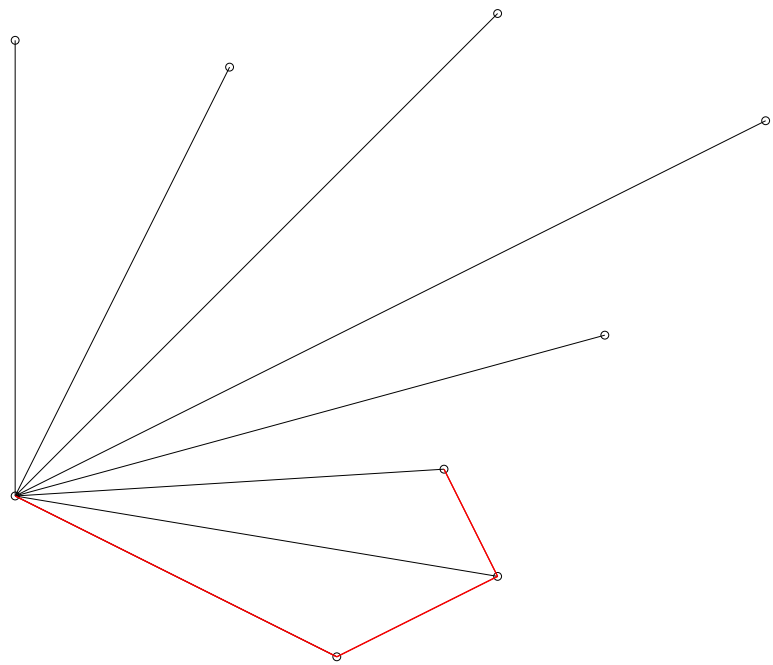


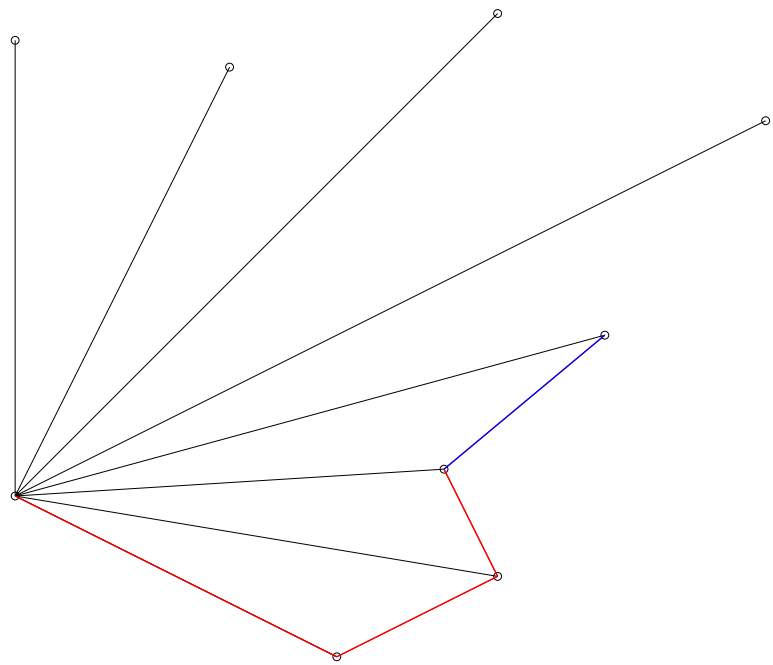
8. ábra. A pontok halmazának rendezése polárszög szerint.

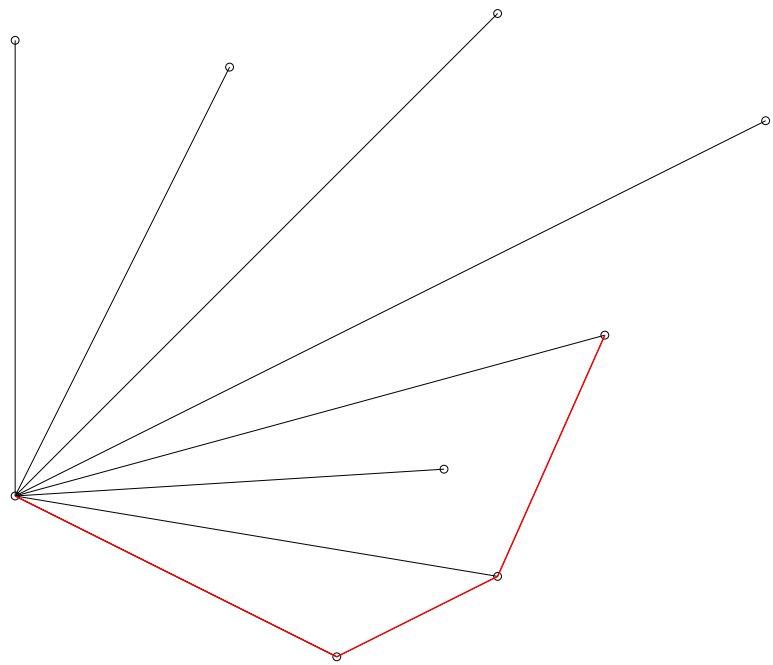


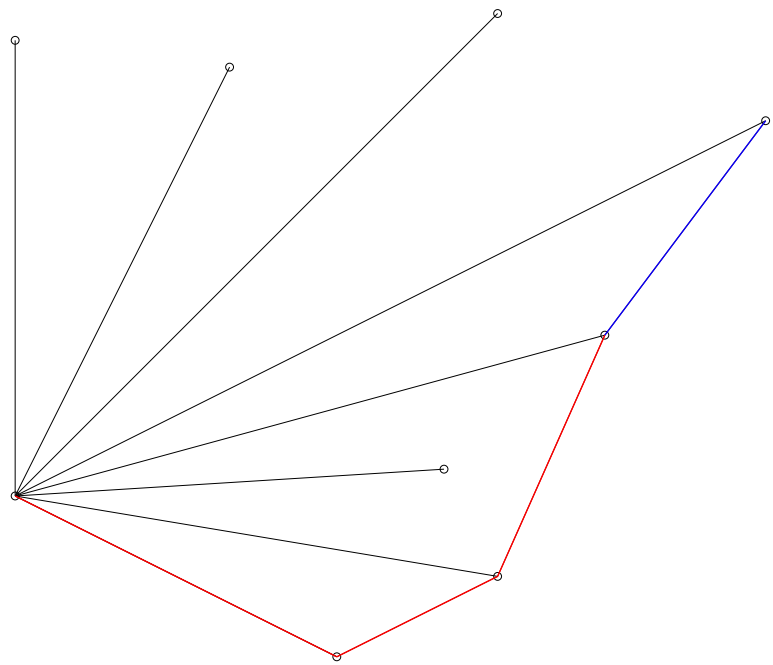
9. ábra. Minden egyenesen csak a legtávolabbi pont marad meg.

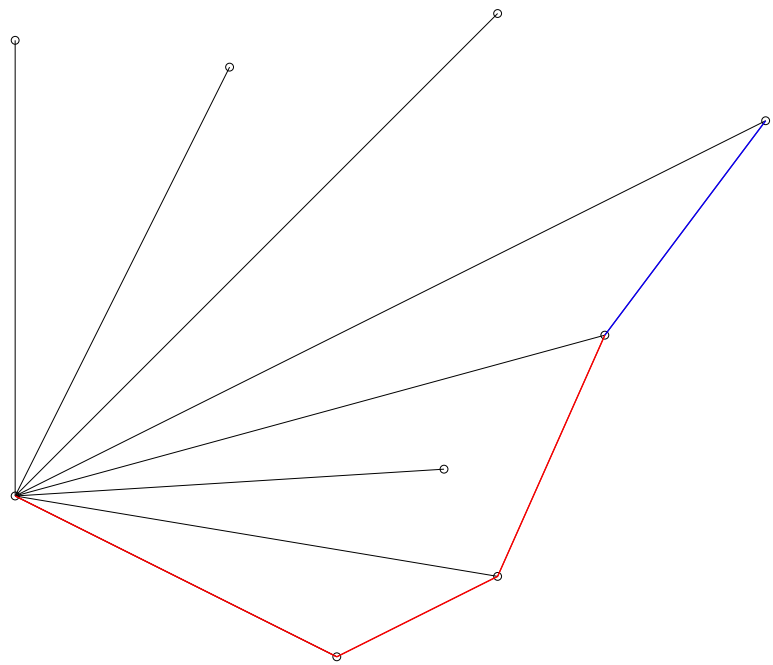


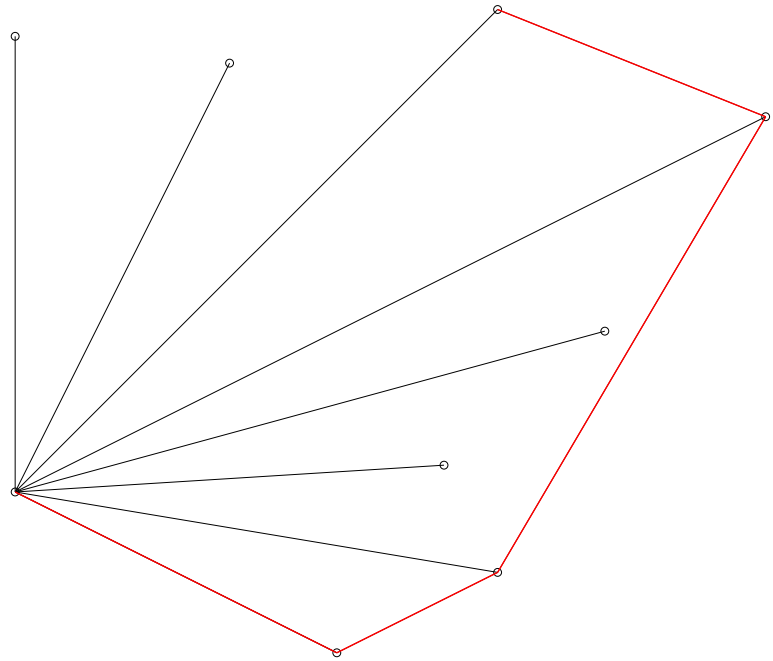












```
void KonvexBurok(vector<Pont>& P, vector<Pont>& Q){
//Bemenet: P[0],...,P[n-1],
//Kimenet: Q[0],...,Q[m] a konvex burok csúcsai }
  int n=P.size(), m,i;
  Q.assign(P.begin(), P.end());
  SarokRendez(Q); //
  m=0;
  for(int i=1;i<n;i++){
    while(m>1 && Fordul(Q[m-1], Q[m], Q[i])<=0)
      m--;
    m++;
    swap(Q[m], Q[i]);
  }//for
  Q.resize(m+1);
}
```

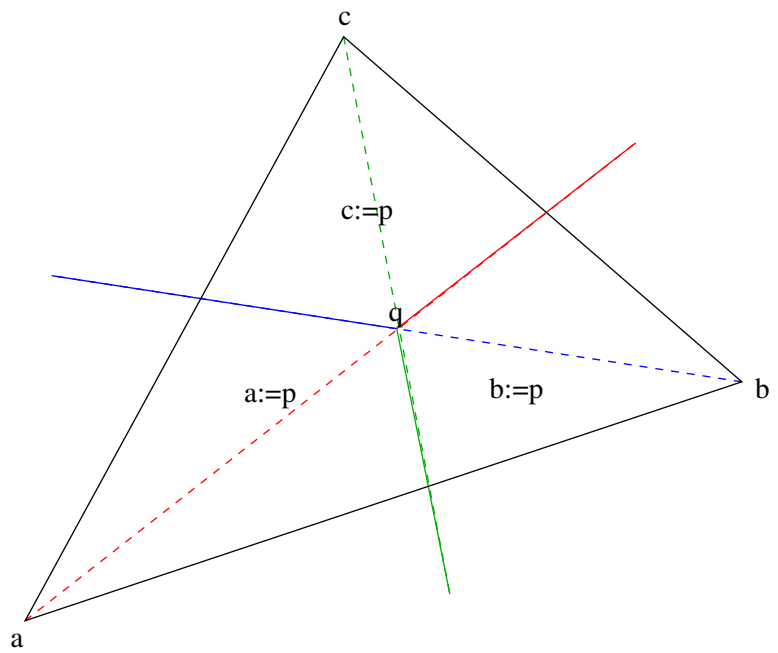
Gyakran elegendő csak a konvex burok csúcspontjainak az azonosítóit megadni.

# 7. Feladat: Bekerítés

Adott a síkon a $P = \{p_1, \ldots, p_n\}$ ponthalmaz és egy $q\,(q \notin P)$ pont. Határozzunk meg három olyan $a, b, c \in P$ pontot, hogy a $q$ pont az $\triangle(a, b, c)$ háromszögbe, vagy oldalára esik, de a $P$ ponthalmaz egyetlen más pontja sem esik a $\triangle(a, b, c)$ háromszögbe, vagy oldalára!

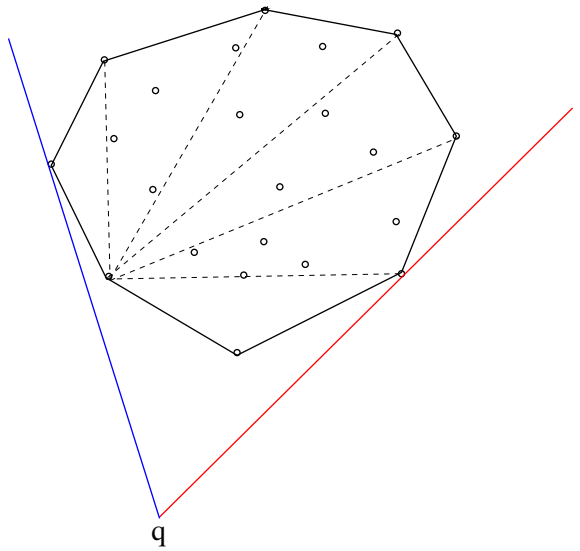
**Megoldás.**

**1. Állítás.** Ha van olyan (tetszőleges) $a, b, c \in P$ pont, hogy $q$ az $\triangle(a, b, c)$ háromszögbe, vagy oldalára esik, akkor ez finomítható úgy, hogy a feltétel teljesüljön.

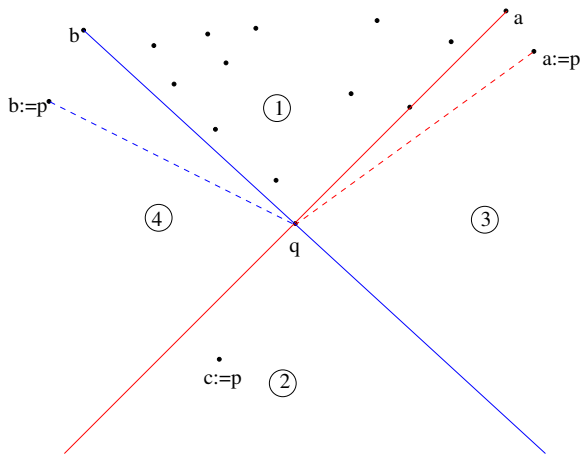



A háromszög finomítása.

**2. Állítás.** Akkor és csak akkor van megoldás, ha a $q$ pont a $P$ ponthalmaz konvex burkán belül, vagy oldalán van.



A konvex burok előállítása nélkül, gyorsan (lineáris időben) kereshető három olyan $a, b, c \in P$ pont, hogy $q \in \triangle(a, b, c)$. Legyen $a := p_1$, majd keressünk olyan $b \in P$ pontot, hogy $a$, $b$ és $q$ nem esik egy egyenesre. Ha nincs ilyen $b$ pont, akkor nincs megoldás.
Ezután minden $p \in P$ pontra ($p \neq a$, $p \neq b$) határozzuk meg, hogy a $(q, a)$ és $(q, b)$ egyenesek által meghatározott síknegyedek melyikébe esik $p$.




Az eddig vizsgált pontok a $\sphericalangle(b, q, a)$ tartományba esnek.

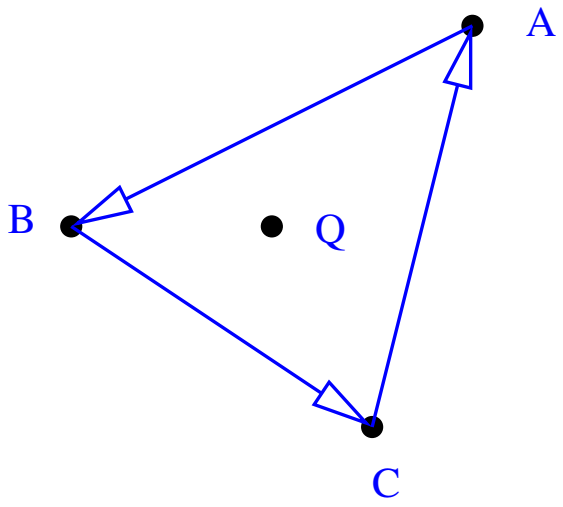
Újabb $p$ pont setén:
1. eset. $\texttt{Fordul}(q, a, p) \geq 0$ és $\texttt{Fordul}(q, b, p) \leq 0$: nem módosul semmi.
2. eset. Egyébként, ha $\texttt{Fordul}(q, a, p) \leq 0$ és $\texttt{Fordul}(q, b, p) \geq 0$: $c := p$ és vége.
3. eset. Egyébként, ha $\texttt{Fordul}(q, a, p) < 0$: $a := p$.
4. eset. Egyébként, (ha $\texttt{Fordul}(q, b, p) > 0$): $b := p$.
Ha nem a 2. esettel ért véget a keresés, akkor nincs megoldás,
mert minden pont $\overrightarrow{qa}$-tól balra és $\overrightarrow{qb}$-től jobbra van.

```
void Bekerites(vector<Pont>& P,  Pont Q, int &A, int &B, int &C){
  int n=P.size(), i=1;  A=0;B=0;C=0;
  while(i<n && Fordul(Q,P[0],P[i])==0) i++;
  if(i==n){return;}
  Pont PA,PB,PC; PC.azon=-1;
  if(Fordul(Q,P[0],P[i])<0){
    PA=P[i]; PB=P[0];
  }else{
    PA=P[0]; PB=P[i];
  }
  int FQAi,FQBi;
  for(int i=i+1;i<n;i++){
    FQAi=Fordul(Q,PA,P[i]);
    FQBi=Fordul(Q,PB,P[i]);
    if(FQAi>=0){
      if(FQBi>0)
      PB=P[i];
    }else{
      if(FQBi<0)
      PA=P[i];
      else{
        PC=P[i]; break;
      }
    }
  }
```

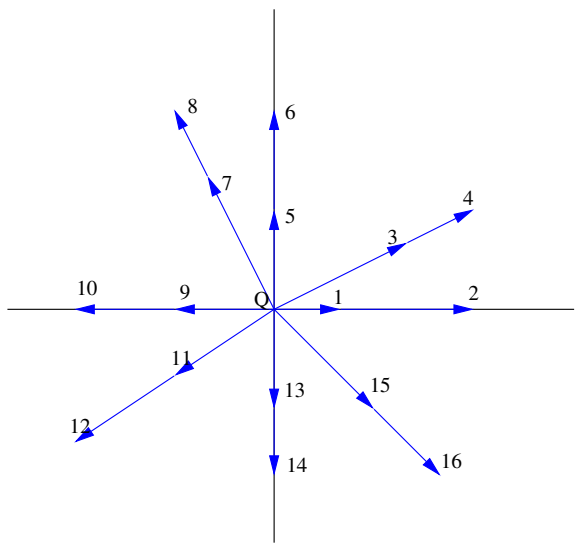
```
1    if(PC.azon<0) return;
2    //háromszög finomítás
3    for(int i=0;i<n;i++){
4      if(i==PA.azon||i==PB.azon||i==PC.azon) continue;
5      if(Fordul(PA,PB,P[i])<0||Fordul(PB,PC,P[i])<0
6                ||Fordul(PC,PA,P[i])<0)
7      continue;
8      // P[i] a háromszögben van.
9      int FAQi=Fordul(PA,Q,P[i]);
10     int FBQi=Fordul(PB,Q,P[i]);
11     int FCQi=Fordul(PC,Q,P[i]);
12     if(FBQi>=0 && FCQi<=0)
13     PA=P[i];
14     else if(FAQi<=0 && FCQi>=0)
15     PB=P[i];
16     else if(FBQi=0 && FAQi>=0)
17     PC=P[i];
18   }
19   A=PA.azon; B=PB.azon; C=PC.azon;
20   return;
21 }
```

## 8. Középpontos rendezés

Látható, hogy adott Q szerinti polárszög szerinti rendezés nem lesz lineáris rendezési reláció, ha a Q pont nem sarokpontja a rendezendő ponthalmaznak.
Az ábrán látható pontokra teljesül, hogy a Q pontra vonatkozó polárszög szerint rendezés esetén $A \leq B$, $B \leq C$, $C \leq A$. A tranzitivitás miatt $B \leq A$ is teljesülne, de $A \neq B$.




Ha a Q pont nem sarokpont, akkor a rendezési reláció megadásánál ezt figyelembe kell venni. A gyakorlatban leginkább olyan rendezésre van szükség, amelyet az alábbi ábra mutat.

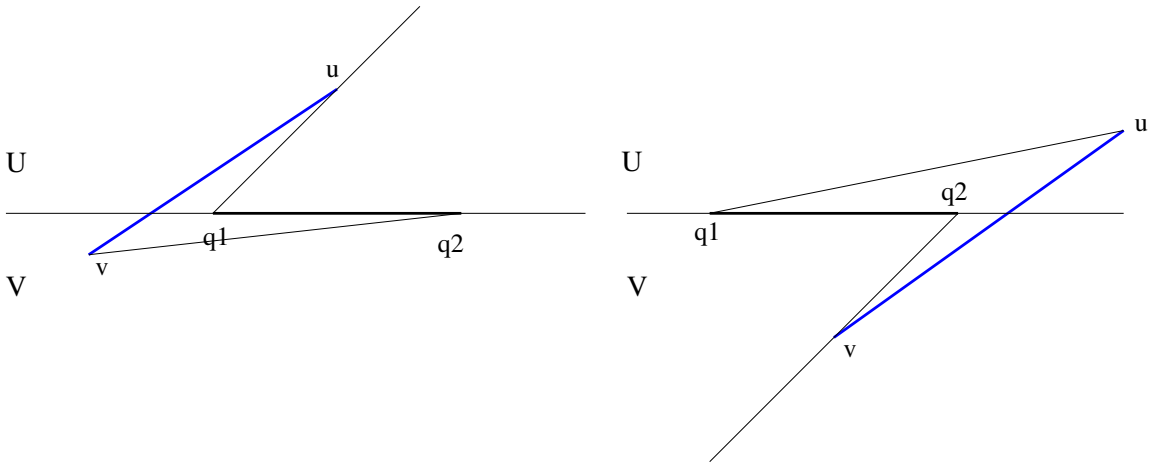



```
bool PolarRend(Pont Q, Pont A, Pont B){
  //A Q ponthoz viszonyított polárszög szerinti rendezésben A és B
  int ford=Fordul(Q,A,B);
  if((A.y-Q.y)>0){
    return ford>0 || ford==0 &&Kozte(Q,A,B);
  }else if((A.y-Q.y)<0){
    return (B.y-Q.y) <0 && (ford >0 || ford==0 && Kozte(Q,A,B));
  }else{//(A.y-Q.y)==0
    return (B.y-Q.y)<0 || ford>0 || ford==0 && Kozte(Q,A,B);
  }
}
```

# 9. Feladat: Metsző szakasz keresése

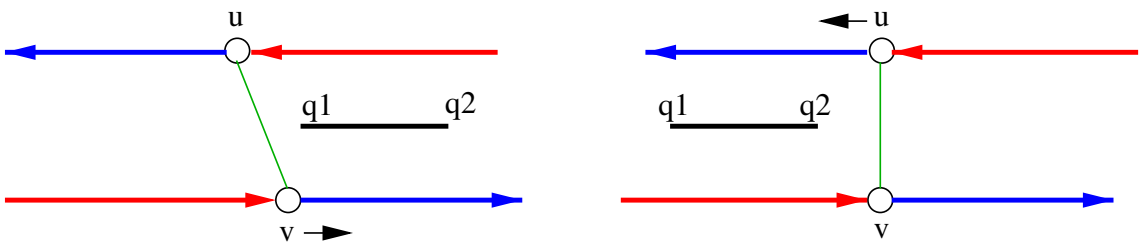
**Bemenet**: `P` ponthalmaz és `q1, q2` pontok (`q1` $\notin$ `P` és `q2` $\notin$ `P` )
**Kimenet**: `p1, p2` $\in$ `P`, hogy a $\overline{\texttt{p1 p2}}$ metszi a $\overline{\texttt{q1 q2}}$ szakaszt.

`O(n log(n))` **futási idejű megoldás**




10. ábra

Legyen `U` a $\overrightarrow{\texttt{q1 q2}}$ -tól balra eső pontok halmaza, `V` pedig a jobbra esőké. Rendezzük `U`-t a `q1` sarokpont szerint, `V`-t pedig a `q2` sarokpont szerint. Ebben a sorrendben haladjunk az `u` $\in$ `U` és `v` $\in$ `V` pontokkal. Ha $\overline{\texttt{u v}}$ és a $\overline{\texttt{q1 q2}}$ nem metszők, akkor vagy `u`-va, vagy `v`-vel kell lépni a rendezési sorrendben.

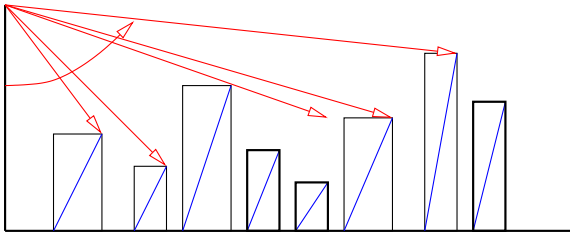



Az ábrán a piros vonalak azokat a pontokat jelölik, amelyeket már vizsgáltunk, és de nincs hozzájuk a másik oldalon olyan pont, amellyel metsző párt alkotnának.

```
void MetszKeres(vector<Pont>& P,Pont q1,Pont q2,
                Pont& p1,Pont& p2){
  vector<Pont> U,V;
  int n=P.size();
  for(int i=1;i<=n; i++){
    if(Fordul(q1,q2,P[i])>0) U.push_back(P[i]);
    if(Fordul(q1,q2,P[i])<0) V.push_back(P[i]);
  }
  SarokRendez(U,q1);
  SarokRendez(V,q2);
  p1=Pont(0,0,0); p2=Pont(0,0,0);
  int pu=0, pv=0;
  while(pu<U.size() && pv<V.size()){
    if(Fordul(V[pv], q1, U[pu])>0){
      pv++;
    }else if(Fordul(V[pv], q2, U[pu])<0){
      pu++;
    }else{
      p1=U[pu]; p2=V[pv];
      break;
    }
  }
}
```

# 10. Házak megvilágítása

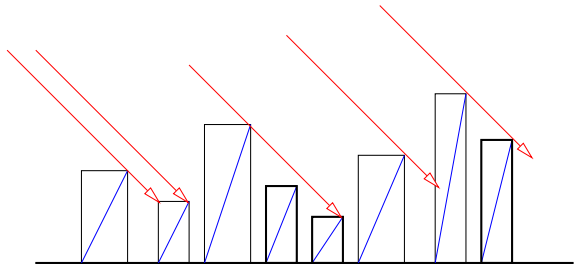
Egy házsor előtt egy lámpa világítja meg az épületeket, amelyek mindegyike téglalap oldalú. A lámpa a legmagasabb épületnél is magasabban van. Adjuk meg azokat az épületeket, amelyek teljesen árnyékban vannak!



**Megoldás:** Egy ház akkor és csak akkor van teljesen árnyékban, ha a jobb-felső sarkát nem éri fény.
Legyen az `L` pont (a síkon) a lámpa helye, $P_i$ pedig az i-edik épület jobb felső sarka.
Az 1. épület jobb-felső sarka biztosan látszik. Vizsgáljuk sorrendben, hogy a $P_i$ pontot megvilágítja-e a lámpa fénye.
Legyen `U` a vizsgálatban az utolsó pont, amelyet megvilágít a lámpa. Ekkor az i-edik épület akkor és csak akkor van teljesen árnyékban, ha `Fordul(L, U,` $P_i$`)` $\leq 0$.
Ha `Fordul(L, U,` $P_i$`)` $> 0$ akkor `U :=` $P_i$

# 11. Házak napsütésben

Egy házsor házaira `dy/dx` szögben süt a nap. Határozzuk meg, hogy melyek azok a házak, amelyeknek legalább egy pontját süti a nap!

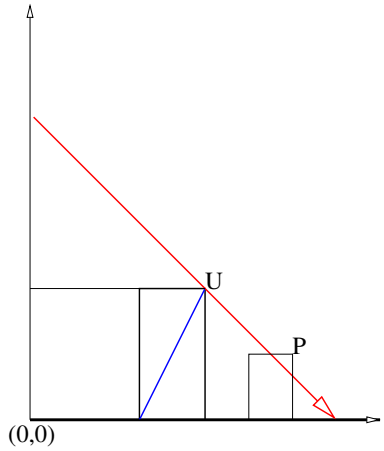


Most is igaz, hogy akkor és csak akkor süti a nap egy ház legalább egy pontját, ha süti a ház jobb-felső sarkát.
A napsugarak párhuzamosnak tekinthetjük. A házsor első háza látszik. Legyen `U` az utolsó olyan ház jobb-felső sarka, amelyet megvilágít a nap. ekkor a következő ( `U`-tól jobbra lévő) házra akkor és csak akkor esik fény, ha

$$\frac{\mathtt{U.y} - \mathtt{P.y}}{\mathtt{U.x} - \mathtt{P.x}} < \frac{\mathtt{dy}}{\mathtt{dx}}$$

$$(\mathtt{U.y} - \mathtt{P.y}) \cdot \mathtt{dx} < (\mathtt{U.x} - \mathtt{P.x}) \cdot \mathtt{dy}$$

Ekkor `P` legyen az új `U` pont.



# 12. Feladat 1: Lefedés két egyenessel, Mester/Geometriai algoritmusok 49

Adott a síkon N pont. A pontokat az 1, ..., N számokkal azonosítjuk.

Készíts programot, amely meghatároz olyan a, b, c és d sorszámú pontokat, hogy a ponthalmaz minden pontja rajta van vagy az a és b sorszámú pontok által meghatározott egyenesen, vagy a c és d sorszámú pontok által meghatározott egyenesen!

## Bemenet

A standard bemenet első sorában a ponthalmaz pontjainak a száma ($3 \leq N \leq 100\,000$) van. A második sorban van a facsemete helyét magadó Fx, Fy koordináták.
A következő N sor mindegyike a ponthalmaz egy pontjának $x_i y_i$ ($-1\,000\,000 \leq x_i, y_i \leq 1\,000\,000$) koordinátáit tartalmazza. A koordináta értékek egész számok.

## Kimenet

A standard kimenetre első sorába a lefedő két egyenest meghatározó a és b, valamint c és d sorszámokat kell írni! Ha nincs megoldás, akkor a 0000 számokat kell kiírni! Több megoldás esetén bármelyik megadható.

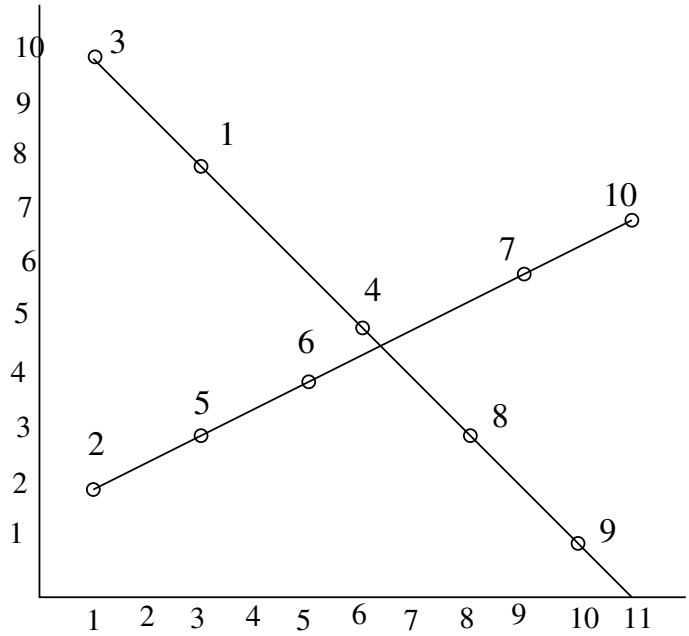
## Példa

Bemenet

```
10
3 8
1 2
1 10
6 5
3 3
5 4
9 6
8 3
10 1
11 7
```

Kimenet

```
1 4 5 6
```




## Korlátok

$3 \leq N \leq 100\,000$
$-1\,000\,000 < x_i, y_i \leq 1\,000\,000$

Időlimit: 0.1 mp.
Memórialimit: 32MiB

## Pontozás

A pontszámok 5%-a kapható olyan tesztekre, ahol $\texttt{N} \leq \texttt{100}$
A pontszámok további 10%-a kapható olyan tesztekre, ahol $\texttt{N} \leq \texttt{1 000}$

# 13. Feladat 2: Facsemete, Mester/Geometriai algoritmusok 39

Egy új facsemetét ültettek a város egyik parkjában, amelyet a kertész meg akar védeni. Azt tervezi, hogy kiválaszt három öreg fát, amelyeket körülkeríti vastag szalaggal úgy, hogy a facsemete szigorúan a szalagok képezte háromszögön belül legyen, más fa pedig ne legyen bekerítve, és a háromszög oldalára sem eshet. A három öreg fa közül egyet már meghatározott.

Készíts programot, amely meghatároz két olyan öreg fát, amelyek a már kiválasztott fával együtt teljesítik a feltételt!

## Bemenet

A `standard bemenet` első sorában a két egész szám van, a park fáinak száma $(3 \leq N \leq 100\,000)$ és a bekerítésre már kijelölt egyik öreg fa sorszáma. Az öreg fákat az $1, \ldots, N$ számokkal azonosítjuk.
A második sorban van a facsemete helyét magadó $Fx$, $Fy$ koordináták.
A következő $N$ sor mindegyike egy öreg fa $x_i y_i$ koordinátáit tartalmazza. A koordináta értékek egész számok.

## Kimenet

A `standard kimenetre` első sorába a megoldást adó két öreg fa sorszámát kell írni! Ha nincs megoldás, akkor a $0\,0$ számpárt kell kiírni.
Több megoldás esetén bármelyik megadható.

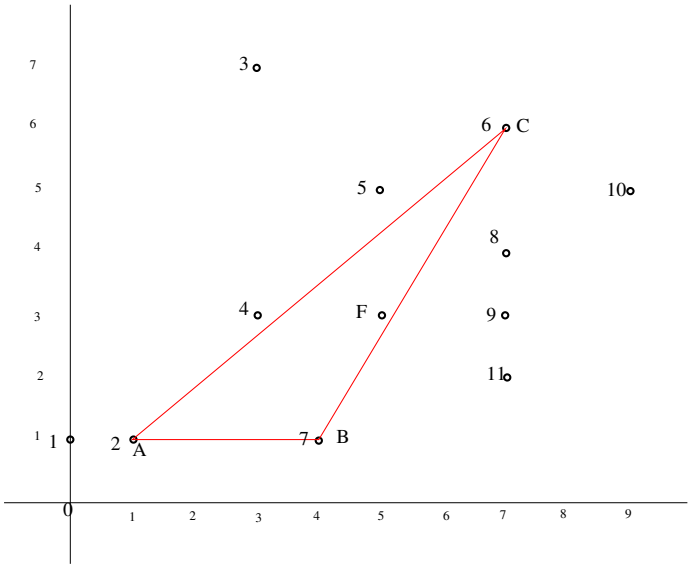
## Példa

Bemenet

```
11 2
5 3
0 1
1 1
3 7
3 4
5 5
7 6
4 1
7 4
7 3
9 5
7 2
```

Kimenet

```
7 6
```




## Korlátok

$3 \leq N \leq 100\,000$
$-1\,000\,000 < x_i, y_i \leq 1\,000\,000$

Időlimit: 0.1 mp.
Memórialimit: 32MiB

### Pontozás

A pontszámok 5%-a kapható olyan tesztekre, ahol $\texttt{N} \leq \texttt{100}$
A pontszámok további 10%-a kapható olyan tesztekre, ahol $\texttt{N} \leq \texttt{1 000}$
A pontszámok további 15%-a kapható olyan tesztekre, nem esik más fa a kijelölt fa és a facsemete által meghatározott egyenesre.

## 13.1. Gyakorló feladatok